National University of Ho Chi Minh City

HCM University of Technology

Computer Science and Engineering Department

Report

Specialized Project

TÊN ĐỀ TÀI

Major: Computer Science

Council: .....

Instructor: Assoc. Prof. Thoại Nam

Secretary: .....

—o0o—

SVTH1: Vũ Hoàng Tùng (2252886)

HỒ CHÍ MINH City, June 2026

Acknowledgement

This section is dedicated to send my most sincere gratitute towards all who have played a role in the completion of this first phase of the thesis. Thanks to that, I was able to complete this thesis gracefully from requirement definitions to system research and development

I would like to show my foremost appreciation to my supervisor, Mr.Thoai Nam for his unwavering support, guidance and constructive feedback. His expertise advice and guidance has really strengthened my vision and understanding, thus been instrumental in shaping the direction of my work.

I am also greatly thankful to Computer Science Falcuty of Department for the learning environment, the facilities, the knowledge and the opportunity to orchestrate this work that I'm proud of. The contributions of my peers and colleagues in engaging discussions and shared insights have been enriching.

To Bảo, a colleague and a friend of mine, whose knowledge and engaging discussion on system design has been a great help, I show my appreciation for his passionate and knowledgable share and consistent heartwarming support.

Finally, to all my friends and seniors who offered encouragement, shared their knowledge, giving inspiration and and especially those who shared years of teaching experience, your collective impact has been indispensible. This work stands firm on the contribution of all those mentioned above, and I are deeply appreciative of the roles each of you played in my orchestration.

DECLARATION OF AUTHENTICITY

I guarantee that this specialized project entitled *"Teaching Assistance System for Study Planning, Recomendation and Management "* was entirely composed and implemented by myself under the guidance and supervision of Assoc. Prof. Thoại Nam at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology.

Throughout the project, I ensure that all references and external sources utilized are appropriately cited and thoroughly documented. Although full code won't be included in report, important parts of the code will be summarizzed in pseudo form and referenced to source of inspiration (if yes).

Ho Chi Minh City, May 2026
The author,

Vũ Hoàng Tùng

Abstract

I like to acknowledge ...

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

The rapid evolution of Artificial Intelligence (AI) has fundamentally change the global socio-economic landscape. To adapt to the new era, selfstudying and learning method are becoming more and more important. Traditional education philosophy, such as ”practice makes perfect,” has no longer been the absolute true; when AI can automate execution, the value of learning moves from procedural fluency and labour expertise to conceptual mastery and strategic analysis. However, current education system fail to keep pace with this evolution:

- **Stagnancy of Content:** Legacy Learning Management Systems (LMS) treat knowledge as static artifacts (textbooks, slides). These materials fail to adapt to the varying cognitive baselines of individual students.
- **The Guidance Deficit:** Without a dynamic roadmap, students navigate a ”sea of information” where external resources are either too granular (leading to information overload) or too condensed (creating knowledge gaps), with little profound ground truth.
- **The Lack of Interaction:** Current systems lack a feedback loop. Textbooks exceed the average human attention span, while slides provide too condensed context insufficient for novices, leaving no room for real-time, human-like inquiry or mentorship.
- **Outdated Knowledge:** Educational materials and methods cannot catch up with the change of technology. Internet are an invaluable resource, however information is chaotic, unstructured and content are confusing to many new learners.

While Large Language Models (LLMs) offer a glimpse into automated resource synthesis and knowledge base construction, significant technical bottleneck prevent their reliable application in complex domains:

- **Semantic Fragmentation in RAG:** Traditional Retrieval-Augmented Generation (RAG) frameworks rely on discrete data chunking. This fragmentation disrupts the global structural integrity of a subject, leading to ”contextual myopia” where the model captures isolated facts but fails to comprehend the overarching hierarchy of the domain.
- **Contextual Saturation vs. Precision:** Expanding the context window of LLMs often introduces diminishing returns. The injection of unrefined data frequently leads to ”contextual overhead” and ”noise,” where the model struggles to distinguish fundamental principles from peripheral details.
- **Relational Limitations of Vector Space:** Relying exclusively on vector embeddings (typically ≈ 1024 dimensions) is insufficient for mapping the non-linear, multi-faceted dependencies inherent in academic taxonomies. Similarity-based retrieval cannot substitute for the deterministic logic required to understand prerequisite relationships.
- **The Reasoning Gap in Pedagogy:** Instruction is a sophisticated, non-linear reasoning task. Current AI systems are largely reactive—proficient in text generation but deficient in long-term goal tracking and proactive guidance. This results in a susceptibility to hallucinations and an inability to execute the ”evaluate-and-guide” workflow essential for effective pedagogy.

1.2 Goals

This project aims to bridge the gap between passive content and active learning by developing a personalized AI-driven educational ecosystem. The specific objectives include:

- **Philosophy of Co-living with AI:** Transforming AI from a simple tool into a pervasive personal assistant that evolves alongside the learner.
- **Cognitive State Tracking:** Establishing a system to record student progress, identify long-term goals, and objectively evaluate theoretical comprehension through interaction.
- **Adaptive Learning Trajectories:** Customizing the learning pace and curriculum dynamically, ensuring that the difficulty level remains within the student's "Zone of Proximal Development."

1.3 Scope

To fulfill these goals, the research focuses on the following technical cores:

- **Knowledge Modeling:** Developing a deterministic "Backbone Graph" that maps semantic connections and context grouping, serving as a structural pattern for LLM reasoning.
- **Context Engineering:** Engineering methods to facilitate reasoning across vast, high-density knowledge bases without losing global coherence.
- **Agent Workflow Orchestration:** Designing multi-agent systems to handle complex, multi-step pedagogical reasoning tasks.
- **Systemic Architecture:** Implementing clean design patterns optimized for scalable, AI-native educational services.
- **User Experience (UX):** Creating a dual-interface system featuring real-time agent streaming for interaction and a comprehensive GUI for administrative and roadmap visualization.

1.4 Thesis structure

There are 7 chapters in this project's report:

- **Chapter 1** introduces the problems, proposes the system's goals, scopes and thesis structure
- **Chapter 2** provides fundamental knowledge for project understand and deep literature review of relevant prior researches covering topic, as well as preview of open-source application.
- **Chapter 3** Requirement Specifications
- **Chapter 4** propose methodologies: we discuss what inferred from previous work and what ways to solve the specific usecase of ours
- **Chapter 5** details the system including: System architecture and its components, settings in each components, and finally, their implementation explanation that fully illustrate how do the system put proposed methodology to practice
- **Chapter 6** analyzes the system outcomes from components to final output as a whole
- **Chapter 7** is the conclusion to the result: what the system achieved, its limitation and future improvements for the final Capstone Project

Chapter 2

Fundamental Knowledge & Related Work

2.1 Fundamental Knowledge

This section discusses the fundamental techniques and applications foundational to the construction of the proposed system, categorized into 3 pillars:

- **Natural Language Processing (NLP):** The foundational process of transforming vast volumes of raw, unstructured textual input into logical, structured data formats that machine can understand. This is fundamental for document processing, reshaping all subsequent cognitive actions.
- **Agentic Orchestration:** The management layer responsible for coordinating specialized AI agents to execute complex, non-linear pedagogical workflows.
- **System Design:** The architectural blueprint that ensures scalability and robustness, utilizing clean design patterns tailored for high-stakes educational environments.

2.1.1 Natural Language Processing

2.1.1.1 Definition

Natural Language Processing (NLP) is a crucial field of study aiming to process unstructured data from documents to a structured, semantically rich representation. This process is designed to harness the full potential of complex entity-relationship paradigms, moving from raw text to machine-inferable logic. As outlined in the core objectives of language understanding, the goal is to put information into a semantically precise form that allows for further inferences.

The standard pipeline for Natural Language Processing follows the sequence:

Information Extraction (IE) $\longrightarrow$ Knowledge Modeling $\longrightarrow$ Information Retrieval

2.1.1.2 Information Extraction (IE)

Information Extraction is the task of identifying and understanding limited, relevant portions of a text to produce a structured representation [5]. Its goal is to filter out linguistic noise and isolate the atomic units of knowledge. Traditional algorithms aim to extract keywords, following thế 2 methods:

- **Keyword and Feature Extraction:** Utilizing dependency parsing and statistical methods (e.g., TF-IDF) to identify high-signal terms based on syntactic importance.
- **Named Entity Recognition (NER):** A critical sub-task used to find and classify concepts or entities (e.g., specific academic theories, formulas, or historical figures) within a text database [5]. This identifies the ”nodes” of our future backbone.

To capture complex entities and relationships that rigid rule-based systems miss, the process relies on probabilistic modeling. The foundation of this approach is Tokenization, which breaks down raw text into atomic units (tokens). Statistical algorithms, such as Conditional Random Fields (CRF) or Hidden Markov Models (HMM), are then applied to these tokens to calculate the probability distribution of various interpretations. This allows the system to resolve linguistic ambiguities and predict the most likely semantic structures or labels for sequences of text. While traditional probabilistic models are effective, they often struggle with sparse data and lack deep semantic understanding. Deep Learning represents a significant advancement by shifting the target towards generating

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Figure 2.1: *Naive Bayes. Calculate probability of next token given n previous tokens using chain rule*

Embeddings. Instead of merely calculating discrete probabilities, Deep Learning architectures map tokens into a continuous, high-dimensional vector space. In this embedding space, geometrically close vectors represent semantically related concepts, allowing the system to grasp the underlying meaning, context, and nuance of the information rather than relying solely on surface-level statistics. The first major breakthrough in applying

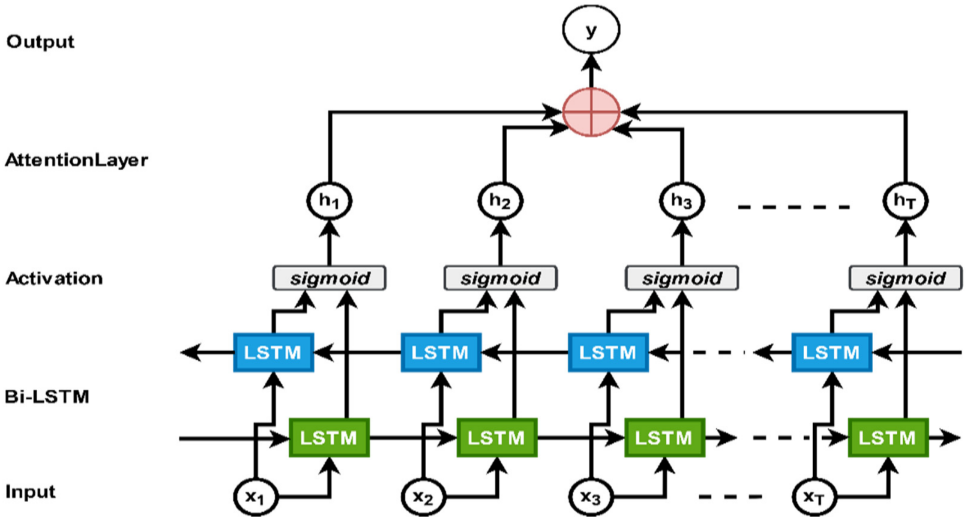

Figure 2.2: *Illustration of general RNN model. Basically a statistical accumulation of previous tokens to output probaility of next token with back propagation*

Deep Learning to text came with Sequential Neural Networks (SNNs), specifically Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks. These models process tokens sequentially—from left to right—maintaining a hidden state that acts as a memory of previous tokens. This sequential processing is crucial for understanding the context of a word based on what preceded it. However, SNNs face limitations in capturing long-term dependencies due to vanishing gradients and are computationally inefficient because they cannot process tokens in parallel. To stabilize gradient vanishing and explosion, LSTM proposes forget gates that

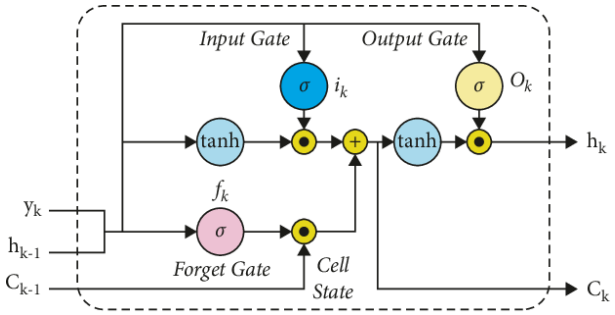

Figure 2.3: *LSTM with forget gate*

The Transformer architecture was introduced, overcoming the drawbacks of SNN and being SOTAs as well as standard method nowadays. It fundamentally change the landscape of NLP and enable modern Large Language Models (LLMs). The core innovation of the Transformer is the Attention Mechanism (specifically, Self-Attention). Instead of processing tokens sequentially, Transformers process the entire sequence simultaneously. The Attention Mechanism calculates a weighted importance score for every token relative to every other token in the sequence, regardless of their distance. This allows the model to instantly capture global context and long-range dependencies,

making it the most powerful method for extracting deep, complex information from educational texts.

2.1.1.3 Knowledge Modeling

Often overlooked in standard RAG systems, Knowledge Modeling is the construction stage where extracted entities are organized into a coherent structure that shapes how the system perceives and reasons.

- **Semantic Parsing and Logical Form:** Mapping extracted relations into a semantically precise form, such as First-Order Predicate Calculus (FOPC) or Knowledge Graphs (KG). This creates a deterministic "Backbone Graph" of the curriculum.
- **Embedding Space Topology:** Modeling knowledge within a continuous vector space where semantic similarity is represented by geometric proximity. This allows the system to handle concepts that are conceptually related but lexically distinct.

2.1.1.4 Information Retrieval (IR)

Information Retrieval is the active execution stage based on the result of construction phases IE and Knowledge Modeling, where the goal is to identify documents or data points relevant to a specific information need from a large document set [5].

- **Lexical and Keyword Matching:** Corresponding to traditional IE, this method uses exact term overlaps to find specific references or symbolic links within the knowledge base.
- **Vector Retrieval:** the most common modern approach, utilizing the embedding space to retrieve context based on semantic similarity, typically calculated via Cosine Similarity:

$$\text{similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} \tag{2.1}$$

- **The GraphRAG Paradigm:** A new powerful emerging method, where retrieval navigates the entity-relationship graph established during the modeling phase. Unlike discrete chunk retrieval, GraphRAG performs "multi-hop" reasoning to connect disparate concepts, enabling the system to resolve complex queries that require a global understanding of the educational roadmap.

2.1.2 Agentic Systems

2.1.2.1 AI Agents: Definition and Capabilities

An Artificial Intelligence (AI) Agent is a computational system capable of autonomous task execution through the dynamic design of workflows using available tools. AI agents can encompass a wide range of functions beyond natural language processing including decision-making, problem-solving, interacting with external environments, and performing actions.

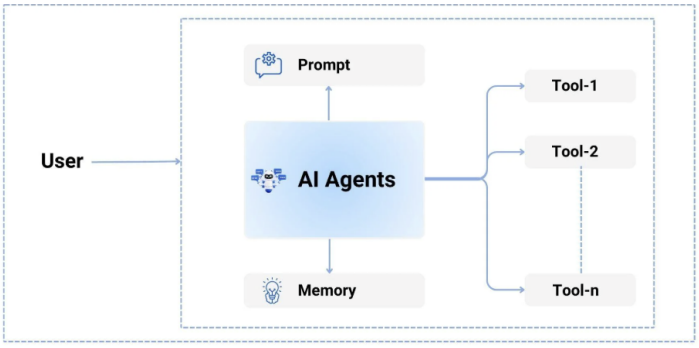

Figure 2.4: *Basic Single Agent diagram*

Unlike Large Language Models (LLMs), which produce responses solely based on the static data used to train them and are bounded by fixed knowledge limits, agentic technology uses tool calling on the backend to obtain up-to-date information. They optimize workflows and create subtasks autonomously to achieve complex goals. It is responsible for translating high-level natural language queries into executable workflows. Instead of relying on rigid, pre-programmed paths, the agent dynamically analyzes the request, identifies the necessary tools, and

orchestrates them in a sequence that best addresses the user's needs. This allows for a more flexible and adaptive system that can handle a wide variety of queries and tasks without requiring explicit programming for each scenario.

What truly pushing agentic systems to achive what LLM can't lies in the harnessing of agentic technology. Harness is the process of constructing and integrating systems or infrastructure that boosts capabilities into the system. The harnessing of agentic system is critical for the agentic performance, as it enables the system to navigate the complex, multi-step processes required to analyze and retrieve relevant video content based on user queries.

LLM: The Logical Core Large Language Models (LLMs) serve as the brain of the agent, represent the thinking capability. They provide the reasoning capabilities necessary to interpret vague user queries and orchestrate the appropriate tools to find answers. The LLM is responsible for parsing the user's intent, identifying missing information, and deciding on the next actions. The reasoning and decision-making depend heavily on the type of LLM used. Upon studying the current industry, mainstream types of models include:

- Reasoning Models: Models such as OpenAI o1 or DeepSeek R1 are optimized for Chain of Thought processing. They excel at complex logical deduction and planning multi-step workflows before taking action. This makes them ideal for the planning phase, where they must decompose difficult user queries into a clear, executable plan before invoking any tools.
- Function Calling Models Models such as GPT-4o or Claude 3.5 Sonnet are fine-tuned specifically to output structured data, such as JSON, rather than just conversational text. This capability is efficient for tool configuration and defining output formats. They are critical for the execution phase, as they can reliably select the correct tools and format the arguments precisely to query the database without syntax errors.
- Multimodal Models Models such as Gemini 1.5 Pro are trained with multimodal input. These models are essential in a video analysis context because they can natively understand both text and visual inputs. This allows the agent to process image frames directly alongside the user's text query without needing a separate description tool to translate visuals into text first.
- Lightweight Efficiency Models Smaller models like Llama 3 8B or GPT-4o-mini are faster and cheaper to run. They are perfect for narrow, repetitive tasks such as summarizing retrieved results or classifying user intent, where the deep reasoning capabilities of a larger model are unnecessary. Moreover, the ability to easily train and fine-tune these models facilitates customization in the design of Agentic systems.

Context Engineering Context engineering, represent the memorization capacity, involves crafting the environment in which the LLM operates to ensure accurate and consistent outputs. This field includes several topics that govern how the model interacts with data and users.

- Prompting Strategies Prompting is the primary method to instruct the agent, helping it understand the goals and constraints it is dealing with. Techniques such as Query Expansion allow the agent to broaden search terms for better recall. Advanced methods like Chain of Thought (CoT) and Language of Thought (LoT) prompt the agent to articulate its reasoning process step-by-step, ensuring that complex logic is handled systematically before a final answer is generated.
- State Management To maintain a coherent interaction, systems must store chat history and the current state. This allows the agent to understand follow-up questions, such as "What happened after that?", by recalling the context of previous turns. State management ensures the agent knows exactly where it is in a multi-step retrieval process, preventing redundant actions.
- Retrieval-Augmented Generation (RAG) Agents cannot remember all the information contained in massive video archives, and this data is often dynamic. Retrieval-Augmented Generation (RAG) is used to retrieve external information that is not stored within the LLM's internal weights. RAG ensures the agent gets the needed information only when requested, querying the vector database to fetch relevant video segments or metadata to ground its answers in factual evidence.

Tool Integration Represent the action capacity, tool integration is the process of connecting external resources (systems, software, hardware) or APIs to the agent, allowing it to perform specific functions that are beyond the capabilities of the LLM alone. This is critical for a automated system, as it enables the agent to interact and manipulate data in real-time. This helps agent truly integrate with the external world, understand its mission from practice instead of static texts, while also pose significant threats of misuse, when agent cause real life mistake and fail to judge. The key to effective tool integration is ensuring that the agent can seamlessly call these tools with the correct parameters and interpret their outputs correctly to inform its next steps.

2.1.2.2 Agent Frameworks

The concept of an Agentic Framework arises from a practical necessity: building reliable autonomous systems requires more than just smart models; it requires a strong structure. Retraining Large Language Models to handle every specific rule or task is extremely expensive and inefficient. Therefore, the industry has shifted towards Agent Frameworks—software architectures designed to control and improve the behavior of AI without changing the model itself.

These frameworks act as a bridge, connecting the creative nature of AI with the reliable logic of software architecture. The Agents are far more than LLMs with tools and context injection; they are intelligent systems with LLMs as the neural core and architecture following software logic. Modern frameworks leverage three main aspects in Agentic design and orchestration:

- Multi-agent Orchestration In complex operations, giving a single agent too much information and too many different tasks often causes errors. A single agent has a limited attention span and context window. Frameworks solve this by using Multi-agent Systems. By breaking a large job into smaller parts handled by different agents, developers can assign specific agents to each task—one for visual analysis, one for audio processing, and one for synthesis—ensuring each agent stays focused and accurate.
- Agent Memorization Intelligent systems need to remember information over time, but standard AI models treat every question as a new conversation (they are stateless). Frameworks solve this by building efficient memory systems with hierarchy, often using classes like Session or Memory. These classes act as data handlers containing multiple attributes and methods to help Agents interact with memory to retrieve previous context during run-time.
- Tool Abstraction Frameworks turn standard code functions into Tools that agents can use on their own. The key difference between a normal function and a Tool is the description: a Tool includes instructions on what it does and when to use it. This allows the framework to give the AI the right capabilities—like a calculator or a database search—allowing the agent to interact with the real world.

2.1.2.3 Langchain and LangGraph

The specific usecase of a our educational system necessitate a flexible, nonlinear, more orchestrated approach. While numerous frameworks exist to harness the capabilities of Large Language Models (LLMs), the intricate requirements of pedagogical guidance necessitate a high-level, state-aware framework.

Framework	Core	Characteristic	Strength in Education	Limitation in Education
LlamaIndex	Data-Centric: Optimizing the interface between LLMs and private data.	Retrieval-heavy; focus on indexing and query engines.	Very powerful when it comes to multistep input output manipulation of tools, LLMs and states	Often prioritizes data retrieval over complex, multi-step pedagogical reasoning.
LangGraph	Explicit state management via cyclic graphs. Supporting edges, nodes creation making full control of logic, branching	**Stateful, nonlinear branching** with fine-grained control	Suitable for complex nonlinear task, with stateaware and context, logic localization and independence	Branching acts as a selling point and a potential logical error, when applying complex flow to simple tasks. Most of all, due to its high level abstraction, input output of LLM get unstable
Google SDK	Tool-Centric: Direct, low-latency access to model capabilities.	Functional/Linear; focus on tool-calling and result passing.	Deliver stable tool results, state and worker result if the pipeline is well designed	Becomes unstable and difficult to maintain when including branching workflows.
CrewAI	Role-Centric: Simulating human-like collaborative teams.	Autonomous, task-based collaboration between predefined roles.	High level, well LLM and human interaction facilitation, capable of complex workflow	Can lack the deterministic, full control orchestration required for strict academic knowledge modeling.

Table 2.1: *Comparison of LLM Orchestration Frameworks*

Unlike traditional chains or data-centric frameworks like LlamaIndex and SDK, LangGraph is built upon a

State Graph Architecture where the system is modeled as a directed graph. In this paradigm, each node represents a specialized agent or a discrete computational step, while the edges dictate the transition logic based on the system's current state. This modularity facilitates a sophisticated level of **Localized Context Management**. Iinstead of burdening a single agent with a big context pile of conversation history, dozen of tools and various educational usecase , the system maintains a shared State object. By granting each agent access only to the specific subset of information required for its immediate task—such as the active knowledge node or the latest student response—LangGraph ensures high-precision reasoning and mitigates the risk of context dilution while maintaining an optimized prompt size.

This granular control over context directly enables Non-linear Pedagogical Branching, allowing the system to mirror the complexity of human instruction. Because the workflow is state-aware, the system can gracefully navigate diverse academic scenarios, utilizing cycles and conditional logic to adapt to the learner's performance. For instance, if a student demonstrates a conceptual misunderstanding, the graph can autonomously trigger a "recovery" branch to revisit foundational prerequisites before returning to the primary roadmap. This deterministic yet flexible orchestration ensures that the AI can handle multifaceted educational use cases without losing track of the student's long-term learning objectives, a feat that is statistically difficult to achieve with stateless or purely autonomous agent frameworks.

Upon studying various framework, our research direction shift to LangGraph since it is the most suitable framework for this educational system. While LlamaIndex offers stable operation with full control data, its data-centered nature often obscures the complex logic and asynchronization required for student-teacher interaction. The Google Generative AI SDK, though efficient for direct tool integration, lacks the complex state-management infrastructure necessary to handle the unstable nature of complex workflow, comparing to predecessor framework. Ability to maintain a persistent state and manage non-linear transitions of LangGraph and CrewAI provides the deterministic backbone required to facilitate a truly adaptive and reliable learning experience.

2.1.3 System Design

This section details the software engineering principles, architectural decisions, and technological stack employed to construct the system, with the purpose of delving deep into concepts and understanding for modularity, scalability, and the ability to handle the asynchronous nature of heavy video processing workflows.

2.1.3.1 Software Paradigm: Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects," which can contain data in the form of fields (attributes) and code in the form of procedures (methods). In this paradigm, computer programs are designed by making them out of objects that interact with one another. Fundamentally, this paradigm relies on four core concepts:

- Class: A blueprint or template for creating objects, defining the initial state and behavior.
- Object: A specific instance of a class, representing a concrete entity within the system memory.
- Attribute: Variables bound to an object that represent its internal state or data.
- Function (Method): Procedures associated with a class that define the behaviors or actions an object can perform.

In the context of System Design, OOP is the dominant industry standard because the core components of this architecture are inherently stateful. Unlike simple functional scripts that execute and exit, system components must maintain persistent internal states—including conversation history, active tool configurations. The OOP provides the natural structural to manage this state lifecycle. The system leverages the four main pillars of OOP:

- Abstraction: This allows the system to model complex entities by hiding internal implementation details and exposing only relevant operations, allowing the outside workflow logic to interact with high-level methods rather than raw data from lower components.
- Encapsulation: Groups related variables and functions into a single unit, protecting the internal state from unintended interference. This ensures that the memory of one component is strictly isolated and cannot be corrupted by the operations of another component running in parallel.
- Inheritance: This promotes code reusability by allowing new classes to derive properties from existing ones. We utilize a base class to define shared logic such as error handling, logging, and connection management. Specialized classes then inherit these foundational capabilities while implementing their specific unique behaviors, significantly reducing code duplication.

- Polymorphism: This allows objects of different classes to be treated as objects of a common superclass. In the orchestration layer, the system can iterate through a diverse list of components—such as different retrieval engines or analysis modules—and trigger their execution loops using a uniform interface. The orchestrator does not need to know the specific type of component it is invoking, only that every component adheres to the expected interface contract.

2.1.3.2 System Architecture: Microservices

System architecture defines the fundamental structure of a software system, organizing its components and their interrelationships to dictate data flow and logic distribution. Given the necessity of building a system responsible of transfering, managing and storing enormous amount of data, this subsection discuss a wellknown scalable system architecture concept used in industry: Microservices.

Microservice architecture is a design paradigm in which an application is organized as a suite of small, autonomous services. Each service operates in its own process and interacts with others through well-defined APIs (Application Programming Interfaces). In contrast to monolithic architectureswhere the entire application is developed and deployed as a single, unified system—microservices support modular development and independent deployment. The adoption of Microservices is driven by the core idea: dividing task into smaller subtask, which

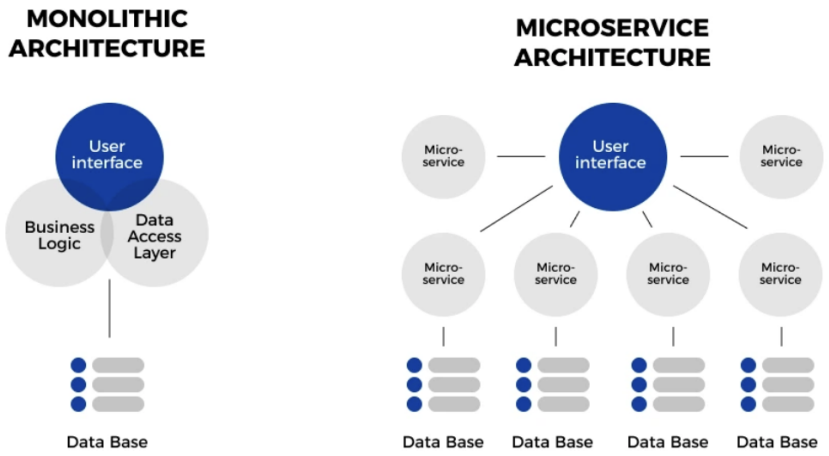

Figure 2.5: *Comparison of Microservice and Monolyth architecture*

bring some benefits to the system's workload:

- Independent Scalability: This is the critical advantage. Microservices allow the system to scale the heavy to high-performance hardware without unnecessarily scaling the lightweight operation like I/O, optimizing resource utilization.
- Fault Isolation: In a distributed system, failures are contained. The rest of the application remains operational, ensuring greater availability.
- Technological Flexibility: Microservices allow each component to use the optimal technology stack. AI agents can use Python for its rich ML ecosystem, while the API gateway can utilize high-concurrency, modular languages like Go, preventing the system from being locked into a single framework.

2.1.3.3 Technology Stack and Frameworks

The selection of frameworks is driven by the need for high-performance Python execution and seamless integration with AI libraries.

Backend Development Backend development serves as the foundational logic of any software application, responsible for data processing, security, and the orchestration of communication between the database and the client. It handles the business logic that users do not see but rely upon for functionality. For a video retrieval system involving heavy AI workloads, the backend must be capable of handling high-concurrency requests without latency, ensuring that the complex computations of the agents do not block the user experience.

To address these requirements, we utilize FastAPI, a modern web framework for building APIs with Python. While older frameworks like Django were built on synchronous standards known as WSGI, FastAPI adopts the

Asynchronous Server Gateway Interface (ASGI). This approach allows the server to handle asynchronous code natively using Python's await syntax. This represents a paradigm shift for AI applications, moving away from blocking, sequential request handling to a concurrent model where I/O-bound tasks—such as waiting for an LLM response or a database query—release the CPU to handle other incoming requests, offering some key features:

- High Concurrency: By utilizing the ASGI standard, FastAPI can handle thousands of simultaneous connections (such as multiple users uploading videos or chatting with agents) without the blocking issues inherent in traditional synchronous frameworks.
- Data Validation: It integrates tightly with Pydantic, a data validation library. This ensures that the complex JSON outputs generated by AI agents are rigorously validated against strict schemas before being sent to the frontend, preventing runtime type errors.
- Performance: Built on top of Starlette, FastAPI offers execution speeds comparable to compiled languages like Go, which is critical for minimizing the latency overhead in the video processing pipeline.

Frontend Development Frontend development is critical as it represents the intersection between the system's logic and human interaction. It focuses primarily on user experience, translating raw data into an accessible, responsive interface. The goal is to render the application elegant, fast, and secure, abstracting the complexity of the underlying video analysis tools from the end-user. With this definition, we selected a library that prioritizes a declarative and component-based approach. React is a JavaScript library for building user interfaces, created by

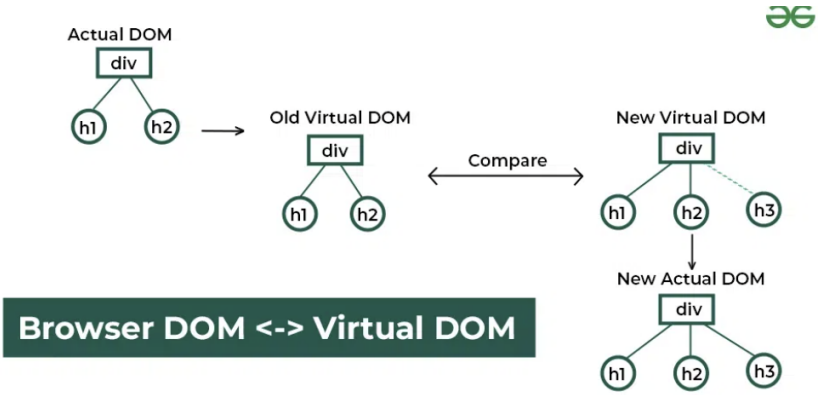

Figure 2.6: *ReAct Virtual DOM; Source: GeeksforGeeks*

Meta in 2011 to address the challenges of building large-scale, dynamic web applications. Unlike previous generations of tools that relied on imperative manipulation of the browser's Document Object Model (DOM), React introduced a declarative model utilizing a Virtual DOM. This is a lightweight memory representation of the UI. When the application state changes, React calculates the difference and updates the real DOM efficiently. This approach allows developers to build complex, stateful interfaces that update in real-time without compromising performance, offering some key features:

- Component-Oriented Architecture: React encourages breaking the UI into independent, reusable pieces called Components (e.g., a SearchBar, a VideoPlayer). This isolation allows for modular development and easier maintenance of the code.
- Unidirectional Data Flow: Data flows in a single direction, from parent to child components. This structure makes the application state predictable and easier to debug, which is essential when managing the streaming data from the AI agents.
- Rich Ecosystem: Being a library rather than a rigid framework, React allows for the integration of specialized tools for routing, state management, and visualization, providing the flexibility needed to build a custom dashboard.

2.1.3.4 Database System Design

Database is where you store necessary information where application can read, write, update and delete via query or integrated calls or methods. In the domain of modern software engineering, traditional monolithic database paradigms are theoretically insufficient for video retrieval. Video data is inherently heterogeneous, characterized by the "3 Vs" of Big Data: High Volume, High Variety, and High Velocity. To address this, robust architectures utilize a Polyglot Persistence strategy, utilizing multiple distinct storage technologies optimized for specific data modalities.

Table 2.2: *Polyglot Persistence Architecture: Strategic allocation of storage engines.*

Storage Engine	Data Modality	System Role	Theoretical Justification
Relational (RDBMS)	Structured Data	User Authentication, Billing, Access Logs	Enforces ACID properties for transactional integrity and supports complex relational joins for administrative analytics.
NoSQL (MongoDB)	Semi-Structured Data	Semantic Metadata, Analysis Tags, JSON Outputs	Provides schema polymorphism to handle variable outputs from different AI agents (e.g., OCR vs. Object Detection) without sparse tables.
Object Storage (MinIO)	Unstructured Binary	Raw Video Files, Extracted Frames, Audio Clips	Decouples storage from compute to prevent database bloat; utilizes Erasure Coding for high-throughput parallel streaming during ingestion.
Vector Database (e.g., Qdrant)	High-Dimensional Vectors	Embeddings Index, Semantic Search	Optimized for Approximate Nearest Neighbor (ANN) algorithms (e.g., HNSW), enabling efficient similarity search in latent spaces where traditional B-Tree indexing fails.

Relational Database (RDBMS)

Relational Databases are mathematically founded on Set Theory and First-Order Predicate Logic. They organize data into tuples (rows) and attributes (columns) within strictly defined relations (tables). The defining characteristic of this model is the enforcement of ACID properties (Atomicity, Consistency, Isolation, Durability), which guarantees that database transactions are processed reliably.

RDBMS is typically utilized strictly for data that requires high integrity and deterministic relationships, such as User Management and Billing. It acts as the source of truth for entity relationships, enabling complex SQL JOIN operations (e.g., selecting all videos uploaded by a specific role within a timeframe).

The adoption of RDBMS for core administrative data offers specific advantages to system stability:

- Transactional Integrity: In a multi-user environment where credits or permissions are updated frequently, RDBMS ensures that race conditions do not occur. For instance, if a user purchases a subscription while simultaneously uploading a video, the ACID properties ensure the account balance is updated accurately before the upload permission is granted.
- Data Normalization: By structuring data into normalized tables (e.g., Users, Roles, Logs), redundancy is minimized. This ensures that a change in a user's details is reflected system-wide instantly, without needing to update thousands of duplicate records.
- Complex Querying: The robust SQL engine allows administrators to perform audit trails and analytics efficiently, such as tracking system usage patterns across different user groups, which is computationally expensive in non-relational systems.

Non-Relational Database (NoSQL)

To address the limitations of rigid RDBMS schemas, Non-Relational databases emerged, often adhering to the CAP Theorem (Consistency, Availability, Partition Tolerance). Unlike RDBMS which prioritizes strong Consistency, NoSQL databases like MongoDB often prioritize Partition Tolerance and Availability, offering a flexible schema-less design.

Document stores are typically the primary engine for storing the Semantic Metadata derived from video analysis. Metadata derived from video analysis is highly variable; one video might contain OCR text, while another contains only Audio Transcripts or Face Recognition data. Document stores handle these as dynamic BSON (Binary JSON) documents, allowing fields to be added on the fly without the sparse matrices typical of rigid SQL tables.

The utilization of a Document Store is critical for the AI-driven nature of video retrieval:

- Schema Agnosticism: AI agents are constantly evolving. If an ”Object Detection” agent is upgraded to output new attributes (e.g., ”Confidence Score” or ”Bounding Box Coordinates”), Document stores allow the new data to be saved immediately without migrating the entire database schema or breaking existing application logic.
- Read Performance for Aggregates: In a VideoQA scenario, the system needs to retrieve the Video Title, Description, Transcripts, and Vector IDs simultaneously. In an RDBMS, this might require joining multiple tables. In a NoSQL system, this data can be stored as a single nested document, allowing the application to retrieve the entire context of a video in a single, high-speed read operation.
- Horizontal Scalability: As archives grow to thousands of hours of video, metadata storage requirements increase. Native sharding capability allows data to be distributed across multiple servers economically, ensuring lookup times remain low even as the dataset grows.

High-Performance Object Storage (MiniO)

While RDBMS and NoSQL excel at storing alphanumeric data, they are fundamentally inefficient for storing massive binary large objects (BLOBs) like video files, leading to database bloat and performance degradation. Object Storage systems like MinIO treat data not as a file hierarchy or a table, but as discrete objects stored in a flat address space.

High-performance architectures utilize Object Storage to handle the High Volume nature of raw video. It utilizes Erasure Coding—a mathematical algorithm that splits data into fragments and expands them with redundant data pieces—allowing for data recovery even if multiple drives fail. This approach decouples storage from compute; the heavy video artifacts are offloaded to Object Storage, while the databases store only the lightweight reference pointers (URLs) to these objects.

Offloading binary data to Object Storage provides the infrastructure necessary for high-throughput video processing:

- Decoupling Storage and Compute: By separating the raw files from the metadata databases, ”database bloat” is prevented. This ensures that the RDBMS remains lightweight and responsive for user queries, while the Object Store handles the heavy I/O load of video streaming.
- Parallel Streaming for AI Ingestion: When an Agent analyzes a video, it requires high-speed access to the file to extract frames. Object Storage supports parallelized byte-range requests, allowing the AI to read multiple parts of a video file simultaneously. This significantly reduces the time required for the ingestion and preprocessing phase compared to reading from a standard file server.
- Cost-Efficiency and Redundancy: Erasure Coding allows the system to survive hardware failures without data loss, while consuming less storage space than traditional replication methods. This is vital for maintaining a massive archive of media files economically.

Vector Database

Traditional databases are designed for structured data and are inefficient for handling high-dimensional vector data generated by AI models. Vector databases like Qdrant are optimized for storing and querying high-dimensional vectors, which represent the semantic content of video segments. They utilize Approximate Nearest Neighbor (ANN) algorithms, such as Hierarchical Navigable Small World (HNSW) graphs, to enable efficient similarity search in latent spaces where traditional indexing methods fail.

The Vector Database is the backbone of the Retrieval-Augmented Generation (RAG) paradigm, allowing the system to perform semantic search based on the embeddings generated by the AI agents. When a user query is processed, it is transformed into an embedding vector, which is then compared against the vectors stored in the database to retrieve the most semantically relevant video segments.

The adoption of a Vector Database is critical for enabling the advanced retrieval capabilities required for video analysis:

- Efficient Similarity Search: Traditional databases cannot efficiently query high-dimensional vectors. Vector databases

- Scalability for Large Datasets: As the number of video segments grows, the vector database can scale horizontally to maintain low latency for similarity searches, ensuring that retrieval times remain fast even as the dataset expands.
- Integration with AI Workflows: Vector databases are designed to integrate seamlessly with AI pipelines
- Support for Dynamic Data: As new videos are ingested and analyzed, their embeddings can be added to the vector database in real-time, allowing the system to continuously evolve and improve its retrieval capabilities without downtime.

2.1.3.5 Deployment and Containerization

Deplloyment is the process of making a software application available for real applicational usage. This section discusses the deployment strategy and the use of containerization to ensure that the system can be reliably and efficiently deployed for our AI pipeline and highperformance stacks as discussed above.

Docker

Docker is a containerization platform that enables developers to bundle applications and their required components into isolated units called containers. Each container acts as a lightweight, virtualized environment that includes all the necessary elements for the application to run. Because containers are portable and consistent, they work reliably across various environments such as on-premise systems, cloud platforms, and clustered infrastructures.

- **Portability:** A major advantage of Docker is its strong portability. Containers package the application together with all its dependencies, making it simple to move workloads between different environments without compatibility issues. Whether running on a local machine or a cloud server, a Docker container behaves the same way, ensuring consistency throughout the development and deployment pipeline.
- **Isolation:** Docker provides strong isolation between applications and their dependencies. Each container operates independently, preventing interference between applications running on the same host. This isolation enhances both security and stability, especially in shared hosting environments.
- **Resource Efficiency:** Docker containers are more resource-efficient than traditional virtual machines. Since containers share the same operating system kernel, they consume far less memory and disk space. This allows more applications to run on a single machine, reducing hardware overhead and simplifying resource allocation.
- **Scalability:** Docker supports efficient scaling of applications. By using orchestration tools such as Kubernetes or Docker Swarm, developers can easily launch multiple container instances and distribute workloads as needed. This flexibility allows systems to rapidly adjust capacity to meet varying levels of demand.
- **Faster Development and Deployment:** Docker improves the speed and efficiency of both development and deployment processes. Developers can work in isolated, reproducible environments identical to production, minimizing integration issues. The deployment process becomes more automated, streamlined, and less prone to errors, saving significant time.

Docker has transformed modern software development by simplifying the creation, deployment, and management of services. Its emphasis on portability, isolation, resource efficiency, scalability, and rapid deployment makes it an indispensable tool for both development and operations teams.

AWS

AWS (Amazon Web Services) is a comprehensive cloud computing platform that provides a wide range of services, including computing power, storage, and databases, all needed for the containerized environment to actually run. Further more, for that running containerized applications to be publicly accessible, it needs to be hosted on a cloud platform that expose accessible domain to the internet. AWS provides that, along with various tools and services for managing and orchestrating containerized applications, and since its foundation it has been developing itself to be the infrastructure that brings:

- **Scalability:** AWS offers a wide range of services that can be easily scaled up or down based on demand. This allows businesses to handle varying workloads without worrying about infrastructure limitations.
- **User-Friendly Interface:** AWS provides a user-friendly web interface and command-line tools that make it easy for developers to manage their resources and deploy applications. This accessibility has contributed to its widespread adoption.

- **Flexibility:** AWS supports a wide variety of programming languages, frameworks, and operating systems, allowing developers to choose the tools that best fit their needs. This flexibility has made it a popular choice for a diverse range of applications.
- **Global Reach:** With data centers located around the world, AWS allows businesses to deploy applications closer to their users, reducing latency and improving performance. This global infrastructure is a significant advantage for companies with an international user base.
- **Security:** AWS provides robust security features, including encryption, identity and access management, and compliance certifications. This focus on security has made it a trusted platform for businesses of all sizes, including those in regulated
- **Reliability:** Among the most wellknown cloud providers such as Google, AWS have a strong reputation for reliability, with such few records of failures (esspecially comparing to Google, from my own experience). AWS has a strong track record of uptime and reliability, with multiple availability zones and data centers designed to ensure high availability. This reliability is crucial for businesses that require consistent access to their applications and data.

AWS has become a dominant player in the cloud computing market due to its comprehensive service offerings, scalability, user-friendly interface, flexibility, global reach, security features, and reliability. It continues to innovate and expand its services, making it a preferred choice for businesses of all sizes looking to leverage cloud technology for their applications and infrastructure needs. Finally, with a big community and ecosystem, AWS provides extensive documentation, support, and third-party integrations, making it easier for newbies to find resources and solutions for their specific use cases. It is a good start to quickly deploy a reliable scalable system without worrying about the underlying infrastructure, allowing the research to concentrate on the core AI and educational system development.

2.2 Related Work

2.2.1 Document-Knowledge Modeling and Retrieval

Converting documents into structured knowledge representations has become a critical task for so long in the industry and education specifically. To replace manual curation in educational contexts, which is time-consuming and not scalable, prior attempts have been focusing on the power of entity extraction, prerequisite relation reasoning, and RAG. This has been indeed a fine strategy, given the success of early supervised models like `PREREQ` [6] using latent topic representations with Siamese networks to infer course dependencies, which achieved over 70 % Precision@100 scores on the `University Course` and `NPTEL MOOC datasets`. However, the traditional methods of entity extraction and relation reasoning often rely on rule-based systems or pure statistical machine learning techniques, which can struggle to capture the complexity and nuance of educational content's relationships. Moreover, recent researches [14, 8] have proven that models fail drastically to access relevant information when documents exceed a certain length. That is the reason why though prior researches gained impressive results on medium retrieval benchmarks, they are not easily applied in industry when it comes to massive and unstructured enterprise data.

To address this, LLM-based frameworks have been widely proposed. A prime example is `KGGen` [9], an automated system that utilizes a clustering algorithm to group synonymous entities and relations, resulting in denser and better-connected graphs that scored an impressive 81.03% average fact accuracy on the `MINE-2` (Measure of Information in Nodes and Edges) benchmark. The `EDC` [15] framework extends this capability by extracting, automatically defining schemas, and canonicalizing knowledge triples without predefined schemas, consistently surpassing state-of-the-art baselines on complex datasets like `WebNLG`, `REBEL`, and `Wiki-NRE`. However, LLMs have deadly drawbacks. First and most popular of all is the "hallucinations problem" leading to inaccurate, destructurized extractions. To mitigate this, `GraphJudge` [4] proposes using a fine-tuned open-source LLM as a "judge" to score and filter out incorrect knowledge triples generated by naive LLMs. Validated on general (`REBEL-Sub`, `GenWiki`) and domain-specific benchmarks (`SCIERC`, `Re-DocRED`), it achieved over 90% judgement accuracy. Furthermore, for large-scale educational and enterprise systems requiring cost and latency optimization, comprehensive hybrid pipelines have proven highly effective in practices. For instance, `EduKGPipeline` [**?**] establishes an end-to-end framework fusing traditional text extraction with transformer-based weighting, demonstrating high precision on educational transcripts from the `CCI dataset`. Similarly, recent efficient GraphRAG architectures [3] substitute expensive LLM extractions with classical dependency parsing to maintain scalability, retaining 94% of LLM-based performance on `RAGAS` metrics when evaluated on real-world enterprise datasets like `CCM` (Custom Code Migration). Finally, `CoDe-KG` [1] utilizes syntactic decomposition to

process complex sentences, thereby significantly enhancing information recall by an 8-point gain on rare relations across the REBEL and WebNLG2 benchmarks.

Another inevitable error lies in the LLMs' limited context length. It is good when discovering short-dependent context/entities/relationships in small sections but fails for long dependencies of extremely long documents and course information. Trying to inject an excessive amount of information even when cut by windows, until now with the current level, only leads to redundant costs from multiple calls, information overload, and decreased accuracy. To uncover hidden relationships and deduce learning paths beyond textual proximity, studies have applied the iterative iPRL [11] method combined with graph-ranking algorithms to learn prerequisite relations without massive labeled data, significantly outperforming baselines by up to 18% in average F1 scores on multi-domain Textbook and MOOC datasets. Notably, as graphs grow, analyzing their macroscopic structure becomes crucial. The Dynamic Knowledge Graph (DKG) [2] approach divides KGs into discrete time snapshots and applies network science to track knowledge evolution. To recommend optimal learning paths, DKG introduces the KC2 (Knowledge Connector Candidate) algorithm, which uses closeness centrality to suggest connections between disconnected knowledge islands. Additionally, to cluster knowledge into logical chapters or modules, the Leiden [12] algorithm is utilized to optimize the CPM quality function, ensuring that knowledge communities are absolutely well-connected and not disjointed—a common flaw of the traditional Louvain algorithm.

2.2.2 Educational Intelligent System

Smart educational systems increasingly leverage AI and educational KGs to personalize learning. This evolution is evident in both academic research and commercial platforms.

In academia, [16] (along with variants like K12EduKG and MOOC-KG) has been developed to uniformly model knowledge from textbooks and online courses. The [**?**] framework automates KG construction from unstructured PDF documents by extracting keywords and linking them to DBpedia. Furthermore, MEduKG pushes beyond text limitations by integrating multi-modal data into the graph, thereby enhancing the system's comprehensiveness and vividness for students.

Researches relating to this field has also reached the market. Popular platforms worldwide like Khanmigo leverage Generative LLMs to provide personalized tutoring and real-time feedback, with enormous available resources. For Vietnamese representative, VioEdu utilizes a manually curated knowledge tree to structure its curriculum, while EdMicro, specialized for enterprises, employs a skill matrix to map out learning objectives and prerequisites. These systems represent different approaches to integrating AI into education, each with its own strengths and limitations in terms of scalability, personalization, and content structuring. Based on practical surveys and evaluation criteria, current educational systems in the market exhibit various characteristics regarding AI and knowledge structuring. A comparative analysis of these platforms is presented in Table **??**.

2.2.3 Conclusion and Research Gaps

During the survey of mentioned representative researches, upon studying from multiple peer reviews and self-reviews of those, this report pinpoints the following core research gaps in the current landscape:

1. **Limitations in Automation and Cost Optimization:** Systems like VioEdu or EdMicro heavily depend on educational experts manually building knowledge trees or skill matrices. On the other end of the spectrum, relying entirely on Generative LLMs (like Khanmigo) to automate the reasoning process is computationally expensive, slow, and highly prone to factual hallucinations, especially when dealing with specialized academic domains.

2. **Lack of Connectivity and Transparency in Lesson Structures:** Current module clustering methods utilized in text-mining often produce disconnected or fragmented knowledge groups. Existing learning platforms typically lack a standard, mathematically proven network algorithm for dividing study modules, leading to disjointed curricula that disrupt the learners' cognitive flow.

3. **Static Roadmap Reasoning and Gap Detection:** While current LMSs and AI tutors provide generated advice, they lack rigorous mathematical network structure calculations to map out prerequisites dynamically. The identification of critical "knowledge gaps" remains static and reactive, failing to automatically trace back to the fundamental bridge concepts (based on network centrality) that cause student bottlenecks.

Criteria	EduKG	Khan Academy	Docebo	VioEdu (FPT)	EdMicro
(Adaptive learning) Personalize the learning process based on needs, abilities, and preferences.	✓	✓	✓	✓	✓
(Intelligent Recommendations) Some personalized suggestions (courses, videos) based on preferences, and areas of improvement.	✓	✓	✓	✓	✓
(Natural language processing for queries) As chat GPT. Based on the questions from learners, it can give out the answer based on the content the LMS has in its DATABASE.	✓	✓			
(Learning analytics/Progress Tracking) AI-based analytics track and analyze user. Can visualize using tracking board, and leaderboard.	✓	✓	✓	✓	✓
(Text-to-audio content) Provide audios for text content.		✓			
(Interactive learning method) Interactive content, such as quizzes, coding exercises, and simulations, can engage learners and reinforce learning.	✓	✓	✓	✓	✓
(Dynamic KG Reasoning & Roadmap) Automatically extract concepts and mathematically reason long-term prerequisite learning paths without manual expert input.					

Table 2.3: *Comparative analysis of representative educational intelligent systems based on key AI and knowledge features.*

Chapter 3

Proposed System and Methodology

3.0.1 Design Pattern

3.0.1.1 Definition

Design patterns are common solutions to problems that often appear in software design. They work like blueprints that developers can adapt to solve issues in their own programs. Unlike ready-made functions or libraries that can be copied directly into code, a design pattern is not a specific piece of code. Instead, it describes a general way to solve a problem, and developers must adjust it to fit their project.

Design patterns are sometimes confused with algorithms because both deal with solving problems. However, an algorithm gives a clear list of steps to follow to reach a goal, while a design pattern is a broader idea that shows how a solution might be structured. As a result, even when applying the same design pattern, the actual code might look different in different programs.

3.0.1.2 Design Pattern in React

In developing modern React applications, employing well-established design patterns can greatly enhance code maintainability, scalability, and clarity. Below are some of the React design patterns integrated into our project to ensure a clean and modular architecture:

1. **Container and presentation patterns**
   In our project, we adopted the container and presentation component pattern to clearly separate concerns between data handling and UI rendering. This design pattern splits components into two distinct roles:

   Container Components: These are responsible for managing data, application logic, and side effects such as API calls or subscriptions. They typically use state hooks (useState, useReducer) and lifecycle hooks (useEffect) to fetch and manipulate data. They do not concern themselves with how the data is displayed.

   Presentation Components: These components are stateless and purely focused on the visual representation of data. They receive data and callbacks as props from their container counterparts and render UI accordingly. This makes them highly reusable and easy to test, as they don't rely on external data sources or internal state.

2. **Component Composition with Hooks**
   Component composition is a fundamental concept in React, and we extended this principle by integrating it with Hooks to create modular and reusable behavior. Instead of repeating logic across components, we encapsulated shared functionality in custom hooks, such as useFormHandler, useAuth, and useProductFilter.

   This approach allowed us to reuse logic across multiple components without duplicating code. Rather than deeply nesting logic or passing down props through multiple layers, hooks provided direct access to logic within each component that needed it. Similar to the container/presentation pattern, hooks helped keep logic out of the UI layer, making components simpler and more focused on rendering.

3.0.1.3 MVC Design Pattern in FastAPI

While FastAPI does not enforce a traditional MVC (Model-View-Controller) architecture, it naturally supports a similar separation of concerns that aligns closely with MVC principles. In FastAPI-based applications, developers typically structure their code into models, routers (or controllers), and views or schemas, creating a clean modular architecture that promotes maintainability, scalability, and clarity.

FastAPI's models are commonly implemented using Pydantic and an ORM (such as SQLAlchemy), where they define the structure, validation rules, and constraints for the application's data. These models represent the domain

entities and handle interactions with the database layer. By isolating data logic in dedicated model components, FastAPI applications maintain a clear boundary between data representation and business logic.

The controller-like functionality in FastAPI is handled by the routing layer. Routers group related endpoints and define how the application responds to incoming HTTP requests. They coordinate the flow of data between models and services, ensuring that input is validated, business rules are applied, and appropriate responses are returned. FastAPI's dependency injection system further enhances the design by allowing controllers to remain lightweight and focused solely on request handling.

Finally, views in a FastAPI context typically correspond to Pydantic schemas used for request and response serialization. These schemas ensure that the data returned to clients follows a consistent and well-defined structure. Because schemas are separate from models, FastAPI applications can control exactly what data is exposed externally, improving both security and maintainability.

Together, this model-router-schema structure achieves the core goals of the MVC pattern: a clean separation of concerns, improved testability, and a codebase that is easier to adapt and extend. Although the framework does not require a strict MVC layout, FastAPI's design naturally supports these architectural principles, making it well suited for both small-scale APIs and complex, enterprise-grade back-end systems.

Chapter 4

Experiment

gay [1]

Bibliography

[1] Sydney Anuyah, Mehedi Mahmud Kaushik, Krishna Dwarampudi, Rakesh Shiradkar, Arjan Durresi, and Sunandan Chakraborty. Automated knowledge graph construction using large language models and sentence complexity modelling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.

[2] Joao T. Aparicio, Elisabete Arsenio, Francisco Santos, and Rui Henriques. Using dynamic knowledge graphs to detect emerging communities of knowledge. *Knowledge-Based Systems*, 2024.

[3] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[4] Haoyu Huang, Chong Chen, Zeang Sheng, Yang Li, and Wentao Zhang. Can llms be good graph judge for knowledge graph construction?, 2025.

[5] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. Stanford University, 3rd edition, 2026. Online manuscript released January 6, 2026.

[6] Chen Liang, Jianbo Ye, Zhaohui Wu, Bart Pursel, and C Lee Giles. Recovering concept prerequisite relations from university course dependencies. In *Thirty-First AAAI Conference on Artificial Intelligence*, 2017.

[7] Chenxi Liu, Yongqiang Chen, Tongliang Liu, James Cheng, Bo Han, and Kun Zhang. On the thinking-language modeling gap in large language models. *arXiv preprint arXiv:2505.12896*, 2025.

[8] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024.

[9] Belinda Mo, Kyssen Yu, Joshua Kazdan, Joan Cabezas, Proud Mpala, Lisa Yu, Chris Cundy, Charilaos Kanatsoulis, and Sanmi Koyejo. Kggen: Extracting knowledge graphs from plain text with language models, 2025.

[10] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[11] Liangming Pan, Chengjiang Li, Juanzi Li, and Jie Tang. Prerequisite relation learning for concepts in moocs. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017.

[12] Vincent A Traag, Ludo Waltman, and Nees Jan Van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific reports*, 9, 2019.

[13] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

[14] Orion Weller, Michael Boratko, Iftekhar Naim, and Jinhyuk Lee. On the theoretical limitations of embedding-based retrieval. *arXiv preprint arXiv:2508.21038*, 2025.

[15] Bowen Zhang and Harold Soh. Extract, define, canonicalize: An LLM-based framework for knowledge graph construction. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9820–9836, Miami, Florida, USA, 2024. Association for Computational Linguistics.

[16] B. Zhao, J. Sun, B. Xu, X. Lu, Y. Li, J. Yu, M. Liu, T. Zhang, Q. Chen, H. Li, et al. Edukg: a heterogeneous sustainable k-12 educational knowledge graph. *arXiv preprint arXiv:2210.12228*, 2022.